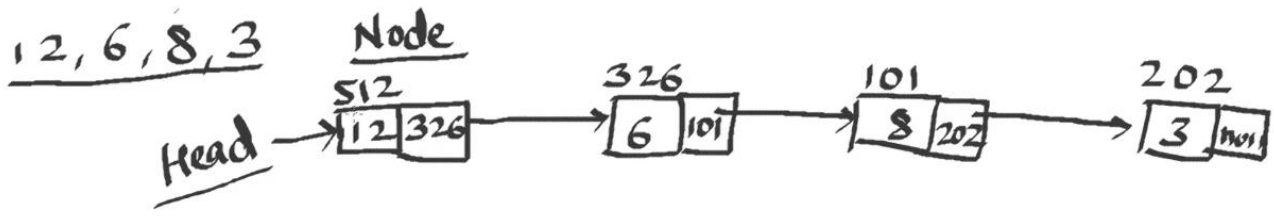
Linked list is a linear data structure as a collection of nodes which contains an information filed and next node's address field. This type of linked list only contains one head (first node in list) and tail (last node in the list) node in entire list, and tail or last node in the list contain null address. And address which is used to access a particular node is known as pointer. Linked list are expandable compare to array so we will be using them accordingly. Traversals can be done in one direction only as there is only a single link between two nodes of the same list.



Indices in linked list work exactly the same as arrays:

Position	Array Index	Linked List Index
First element	`arr[0]`	`head` (index 0)
Second element	`arr[1]`	`head.next` (index 1)
Third element	`arr[2]`	`head.next.next` (index 2)
Fourth element	`arr[3]`	`head.next.next.next` (index 3)

Note: Whenever we have to traverse linked list, in this case singly linked list, we should start from **head** node always.

```
Before: [10] → [20] → [30] → [40]
Index:    0      1      2      3

After inserting 25 at index 2:

Result:  [10] → [20] → [25] → [30] → [40]
Index:    0      1      2      3      4
```

The element that was previously at that index moves to the right (index+1).

What happened to original index 2 element (30)? It shifted RIGHT to become index 3.

Note:

head = [0x7F3A] (points to Node at address 0x7F3A)

After current = head:

Node current = head;

current = [0x7F3A] (now ALSO points to the same Node)

This is copying the address from head to current. Both variables now point to the exact same node in memory.

current = [0x7F3A] (some valid memory address)

null = [0x0000] (special address meaning "nothing")

while (current != null)

Is 0x7F3A != 0x0000? → YES

This is comparing the address stored in current against the special null address.

Statement	What's Being Used	What It Means
`current = head`	The **value** inside `head` (an address)	Copy that address into `current`
`current != null`	The **value** inside `current` (an address)	Compare that address to null

Why `current` Works with Addresses

Because `current` IS an address-holder. That's its ONLY job.

```java
current = head;        // Copy one address to another
current != null;       // Check if address is valid
current.data;          // GO TO that address and get the data
current.next;          // GO TO that address and get the NEXT address
```

Because current was **designed to hold addresses**, not data. That's what reference variables do. The data is inside the node, not inside current. You need the dot (.) to cross from the address to the data.

current tells you WHERE the node is. .data tells you WHAT the node has.

List: `[10] → [20] → [30] → null`

Using `while (current != null)`**:**

```text
Start: current at [10] → not null → process
Next:  current at [20] → not null → process
Next:  current at [30] → not null → process
Next:  current at null → STOP

✅ Processes ALL nodes (10,20,30)
❌ At last node, current.next is null but current itself is valid
```

Using `while (current.next != null)`**:**

```text
Start: current at [10] → current.next is [20] (not null) → process
Next:  current at [20] → current.next is [30] (not null) → process
Next:  current at [30] → current.next is null → STOP

✅ Stops AT last node (30 is not processed!)
✅ Useful when you need to stop at last node (like deletion)
```

- current != null → "Is this node real?"
- current.next != null → "Is there a node after this one?"

current moves from holding one address to holding the next address, which it gets from the node itself.

Memory:

Node1 at address 100: [data=10 | next=200]

Node2 at address 200: [data=20 | next=300]

Node3 at address 300: [data=30 | next=null] ← null means address 0000

head = 100 (head points to Node1)

Code:

```
Node current = head;        // current = 100
while (current != null) {   // Is 100 != 0000? YES
    System.out.println(current.data);
    current = current.next;  // current = 200  }
```

Iteration 1:

current = 100

Check: 100 != null? → YES (100 is a real address)

Print: Go to address 100, get data = 10

current = current.next → Go to address 100, get next field = 200

Now current = 200

Iteration 2:

current = 200

Check: 200 != null? → YES

Print: Go to address 200, get data = 20

current = current.next → Go to address 200, get next field = 300

Now current = 300

Iteration 3:

current = 300

Check: 300 != null? → YES

Print: Go to address 300, get data = 30

current = current.next → Go to address 300, get next field = null (0000)

Now current = 0000 (null)

Iteration 4:

current = 0000 (null)

Check: 0000 != null? → NO

Loop stops

Node next	A variable INSIDE a Node that holds the address of another Node object
current	A variable OUTSIDE that holds address of a Node object

The Two Types of Address Variables

1. Node Variables Inside Objects (`node.next`)

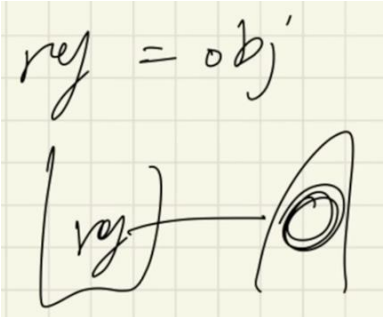
```java
Node node1 = new Node();  // node1 = 100
node1.next = node2;       // Stores address 200 INSIDE node1
```

2. Reference Variables Outside (`current`, `head`)

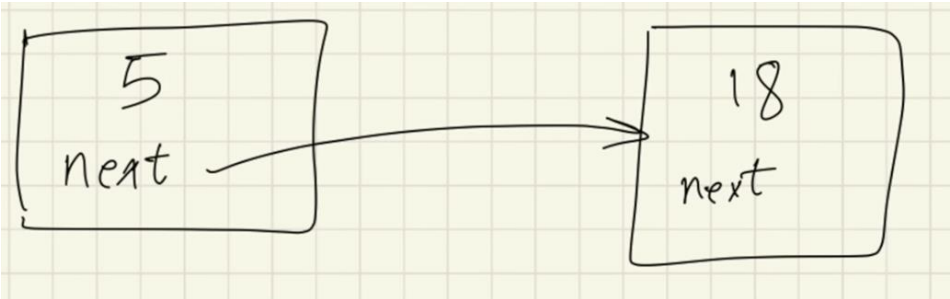
```java
Node current = head;      // current now holds 100
current = current.next;   // current now holds 200 (got from node1.next)
```

1. **References hold addresses**
2. = **assigns addresses between variables**
3. != null **checks if address is real**
4. .next **gets address stored inside a node**

Very important point to understand about reference variable and object , is reference variable in not like pointer but works similarly in Java programming.



Reference variable in Stack memory pointing to object in Heap memory.



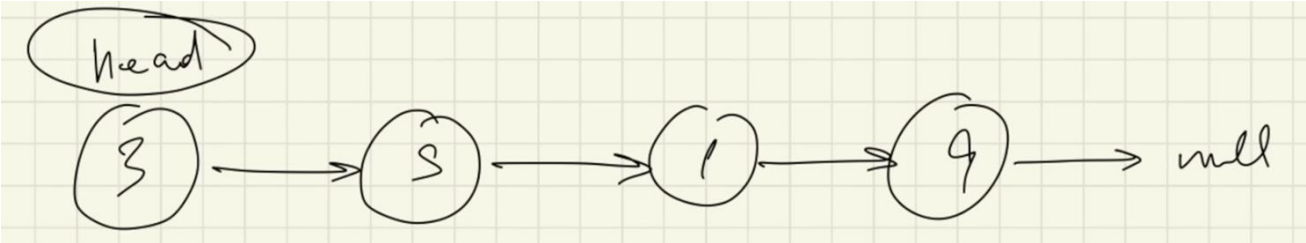
It is amazing work how one reference variable can behave like Node itself but actually it's not. It's like wrap up thing where :

Node n1 = new Node(5); Node n2 = new Node(18);

n1.next = address_of_n2;// pointer to n2 (node).

And by default the value of node reference variable will be null, because it is not pointing to any object.

Example: Node n3; // just declaring not assigning any object



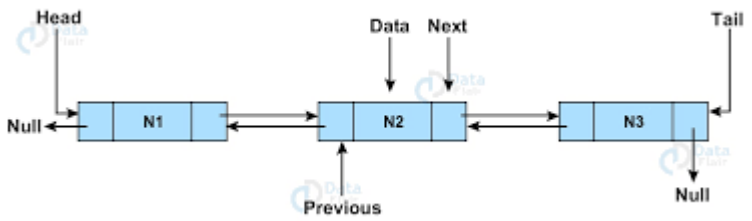
Here we can actually say that tail node or last node is looks something like this:

Node.data = 9;

Node.next = null;// it is pointing nowhere

And note that while traversal we won't and shouldn't use the actual pointer head, instead we use temp node.

What is Doubly Linked List : Is just another type of Linked list where bi-directional traversal is possible, comparing to singly linked list where only one sided traversal (left – right) was possible.

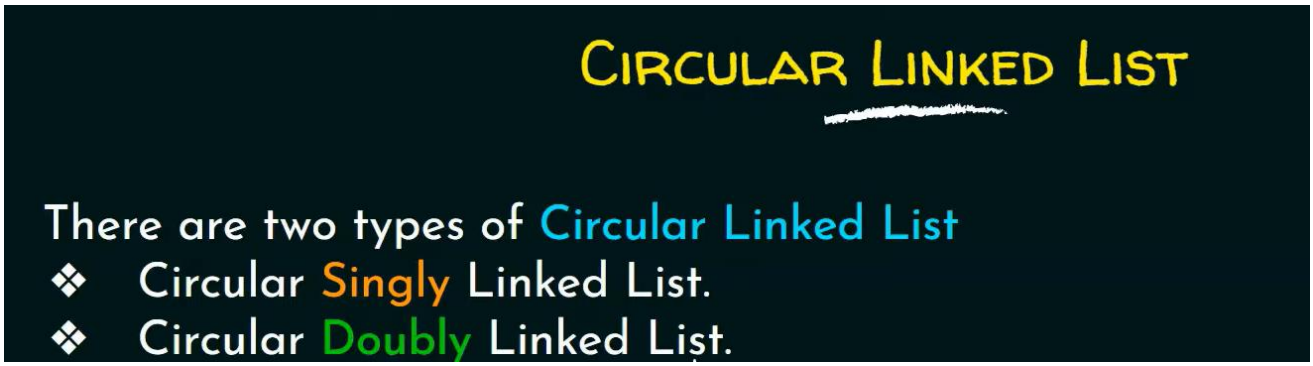



```
Node.java ×
1   package Doubly_Linked_List;
2
3   public class Node {  4 usages  new *  10 related problems
4       Node prev_node;  1 usage
5       int value;  1 usage
6       Node next_node;  1 usage
7       Node(Node prev_node, int value, Node next_node){
8           this.prev_node = prev_node;
9           this.value = value;
10          this.next_node = next_node;
11      }
12  }
```

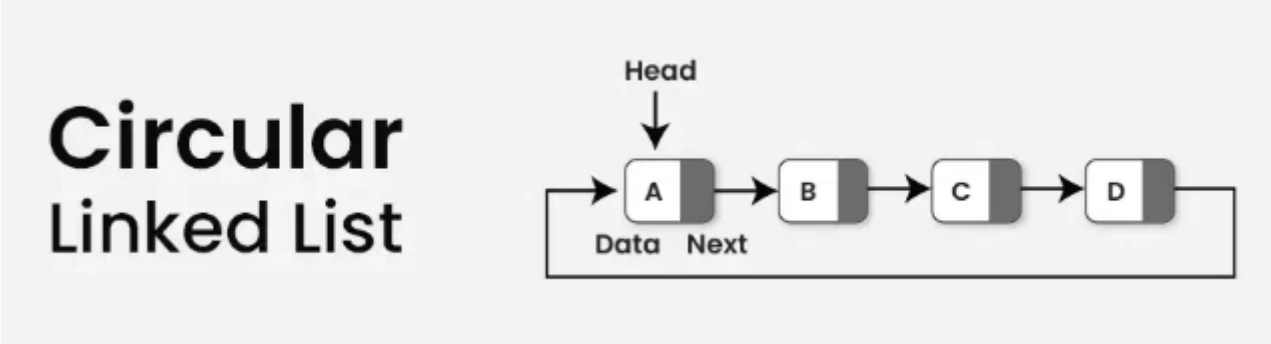
There won't be any difficulty in understanding doubly linked list if we properly grasp how even singly linked list works, difference is just of two reference pointer contain in one node in doubly linked list so for insertion, deletion and traversal we have to make do modification between such pointer ie, next node pointer and previous node pointer.

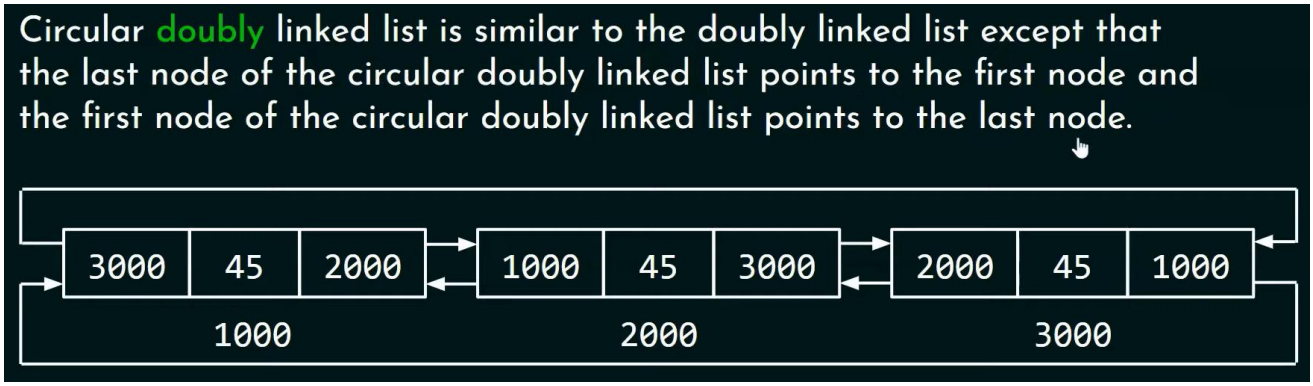
Note: One more thing to mention is, if we have head and tail pointer along with size pointer for linked list it will be much more easier for performing operation so smoothly. But sometimes there can be challenges so we should be capable of performing all operation only having a head pointer.

A **circular linked list** is a variation of a standard linked list in which the last node points back to the first node (head) instead of pointing to NULL, thus forming a continuous loop or a circle.



Circular Singly linked list:





For implementation check out link:

https://github.com/Sandesh775/DSA_BCA_Course/tree/main/Data%20Structures/Linked%20List